PROJECT-BASED LEARNING

PHASE-II EVALUATION

Urban Emergency Pre-emption Engine (UEPE)

Team ID: DSCPP-III-2026-T319 | Team Avengers

Department of Computer Science & Engineering
Graphic Era (Deemed to be University), Dehradun
Academic Session 2026–27

Team & Mentor Details

Team Information

Team ID: DSCPP-III-2026-T319
Team Name: Avengers
Semester: 3rd
Domain: Smart Traffic Management

Team Members

1. Mohit Singh Basera – 2027920 (Team Lead)
2. Pranav Pandey – 2027984
3. Riddhi – 2028472

Mentor

Ram Ji Chauchan

Interactions completed: _____

Phase-I Recap & Progress Since

Phase-I Status

Phase-I identified the problem (emergency vehicles lose critical time at traffic signals), set the objectives, identified the concepts needed, and produced the Phase 01 and Phase 02 architecture diagrams.

Achieved in Phase-I

- Problem and objectives defined
- Concepts identified, working understanding built
- Architecture diagrams for Phases 01 and 02

Carried into Phase-II

- Modular build: Locator, Router, Pre-empt controller
- Simulation first, before any real city data
- Code written for the modules

Problem Statement & Finalized Objectives

Problem Statement

In urban areas, traffic and red lights increase the delay of ambulances, fire trucks and police vehicles. The goal is to give them priority at intersections while keeping normal traffic flowing for other road users.

Finalized Objectives

1. Locate the emergency vehicle and decide its direction, location and priority
2. Predict the fastest route using a shortest-path method (Dijkstra / A*)
3. Turn upcoming signals green ahead of the vehicle, then restore normal operation
4. Stay safe: no conflicting greens, a priority rule for overlapping emergencies

Updated System Architecture / Workflow



Major Components

- Locator – incident and vehicle position
- Router – shortest-path route and ETA
- Pre-empt controller – signal override and restore

Changes Since Phase-I

- Core engine split into three separately tested modules
- Virtual signals and test dashboard for a partial demo
- Simulation before real city integration

Module Design & Functional Requirements

Major Modules

1. Locator
2. Router
3. Pre-empt Controller
4. Light Simulator & Test Dashboard

Functional Requirements

- Accept mock alerts and GPS feed
- Compute best route to the incident
- Switch signals ahead to green, then restore
- Resolve overlapping emergencies by priority

Module Dependency

Locator output feeds the Router; the Router's route and ETA feed the Pre-empt controller; the Pre-empt controller drives the virtual signals shown on the test dashboard.

Technology Stack & Development Methodology

Technology Stack

- Programming: C (data structures) + C++ (OOP)
- Data: in-memory, simulated CSV inputs
- Compiler: GCC / G++
- Dev Tool: VS Code / Code::Blocks
- Version Control: Git / GitHub

Development Methodology

1. Requirement Analysis – done
2. System Design – done
3. Module Development – code written
4. Integration – pending
5. Testing & Validation – pending

Implementation of PBL Course Concepts

Course	Concepts Implemented in Code	Project Module
Data Structures in C	*Graph (adjacency list), binary min-heap / priority queue, hash table*	*Router, Pre-empt Controller*
OOPs with C++	*Classes: Intersection, EmergencyVehicle, SignalController, PreemptionManager*	*All modules (OOP layer)*

Note: Operating Systems and DBMS concepts are not implemented in this project, so they are not claimed.

Development & Implementation Progress

Module	Status	Work Completed
Locator	Code written	*Module code written; not yet tested*
Router	Code written	*Shortest-path (Dijkstra / A*) code written; not yet tested*
Pre-empt Controller	Code written	*Signal override and restore code written; not yet tested*
Test Dashboard	In progress	*Partial prototype to be finished*
Integration & Testing	Pending	*Combine modules, test edge cases and failures*

Overall: module code written; testing, integration and prototype pending.

Working Prototype / Demonstration

Implemented in Code

- Locator module
- Router with shortest-path method
- Pre-empt controller logic
- Class / module organisation

Prototype / Output

Code is available in the GitHub repository. Screenshots and the test dashboard demo will be added once module testing is complete.

Repository: github.com/Riddhi5101/PBL

Testing, Results & Validation

Test / Module	Expected Result	Observed	Status
Locator	*Correct incident / vehicle position*	*Not run yet*	Pending
Router	*Valid shortest route and ETA*	*Not run yet*	Pending
Pre-empt Controller	*Green ahead, then normal restore*	*Not run yet*	Pending
Overlap / failure cases	*Priority rule, no conflicting greens*	*Not run yet*	Pending

Results / Validation

No results yet. Planned metrics: response time with vs without pre-emption, signal-change delay, and conflict-free green checks.

GitHub Repository & Development Activity

Repository Information

- GitHub: github.com/Riddhi5101/PBL
- Module code uploaded
- README describes UEPE
- All members to be added as contributors

To Maintain Before Phase-III

- Regular commits
- Meaningful commit messages
- Individual contributions visible
- Latest implementation and docs uploaded

Requirement: every team member commits individually from their own GitHub account.

Individual Contribution

Member	Role	Technical Contribution	Commits
Mohit Singh Basera	*Team Lead*	*Module code; Phase 01/02 architecture and planning*	*See GitHub*
Pranav Pandey	*Developer*	*Module code; concepts and working understanding*	*See GitHub*
Riddhi	*Developer*	*Module code; GitHub repository*	*See GitHub*

Individual Evaluation

Every member will explain and demonstrate their own code during the review.

Challenges, Remaining Work & Phase-III Roadmap

Challenges Faced

- No real-time traffic, GPS or signal data
- Integrating modules with low latency
- Safe signal changes, no conflicting greens
- Multiple emergencies at one junction

Solutions / Learning

- Simulation first with mock alerts and virtual signals
- Modular build; learning topics along the way

Phase-III Roadmap

1. Finish prototype and test dashboard
2. Integrate all modules
3. Test modules and edge cases (overlaps, failures)
4. Reduce response time, fix errors
5. Test report, final report, presentation

Expected Outcomes & References

Expected Final Outcomes

- Working engine: locate, route, pre-empt, restore
- Measurable drop in response time (with vs without pre-emption)
- Fail-safe mode and priority rule for overlapping emergencies
- Faster, more consistent response; fewer junction accidents

References

1. Cormen et al., *Introduction to Algorithms*
2. Deitel & Deitel, *C++ How to Program*
3. GeeksforGeeks – graphs, heaps, hash tables
4. GTT Opticom – emergency vehicle pre-emption
5. FHWA / ITS JPO – signal pre-emption guidelines

Thank You

Questions & Suggestions

Team Avengers | DSCPP-III-2026-T319 | Mentor: Ram Ji Chauchan